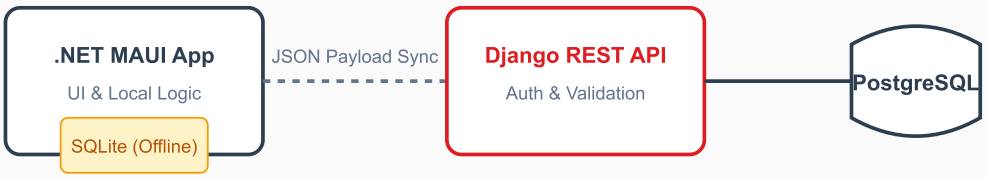
HSE Field Reporter AI Implementation Guide

Target Architecture: .NET MAUI (Frontend), SQLite (Local), Django REST Framework (Backend), PostgreSQL (Master DB).
Project Context: Built to demonstrate offline-first engineering for Shell's "Goal Zero" initiative. Focus on environments with poor connectivity (e.g., offshore rigs, Port Harcourt field sites).

> **Project Scope:** End-to-end mobile hazard reporting app with local offline caching and automated network-restore synchronization.
> **End Users:** Field technicians and HSE officers at remote facilities (e.g., Bonga FPSO, Bonny Island).
> **Target Audience:** Shell HR and Technical Hiring Managers in Port Harcourt.
> **Expected Outcome:** A functional MVP validating alignment with Shell's *Goal Zero* mandate, proving the developer can solve the critical business problem of safety reporting in zero-connectivity zones.

1. System Architecture Blueprint



2. Relational Database Schema (UUID Core)

CRITICAL AI INSTRUCTION: Do NOT use auto-incrementing integers for Primary Keys. Use UUID4 for all models. This is mandatory for offline conflict resolution. Generate UUIDs on the client device at the moment of creation.

Table	Field	Data Type	Description / Constraint
Employee	EmployeeID	UUID (PK)	Unique identifier for staff
	FullName	String(100)	Worker's full name
	Role	String(50)	E.g., "Drilling Engineer", "HSE Officer"
SiteLocation	SiteID	UUID (PK)	Location identifier
	FacilityName	String(100)	E.g., "Bonga FPSO", "Port Harcourt Refinery"
	Region	String(50)	Offshore vs Onshore
HazardReport	ReportID	UUID (PK)	Generated locally via .NET MAUI
	SiteID & EmployeeID	UUID (FK)	Foreign keys referencing tables above
	Description & RiskScore	Text, Int	Details, and severity (1-5 scale)
	IsSynced	Boolean	Local SQLite ONLY. Not needed in PostgreSQL.

> **Mock Data Generation Note:** When generating seed data, utilize relevant regional contexts to stand out. Use locations like *"Bonny Island Terminal"* and *"OML 29"* to show local operational awareness

3. API Design & The Sync Payload

AI INSTRUCTION: The Django backend must expose a bulk synchronization endpoint. Mobile clients in remote locations will bundle multiple offline reports into a single JSON array when network connectivity is restored. The endpoint must parse this array, validate foreign keys (Employee/Site), and perform a bulk insert.



Expected JSON Batch Payload

Ensure the Django REST Framework serializer expects a list of objects exactly matching this schema structure. The `created_at` timestamp must reflect the moment the event was logged locally, not when the API received it.

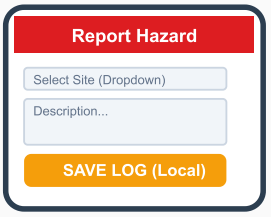
```
      POST /api/v1/hazards/sync/ Content-Type: application/json Authorization: Bearer
       { "device_id": "8f4c-12b2-a98d", "reports": [ { "report_id": "550e8400-e29b-41d4-a716-
446655440000", "employee_id": "f47ac10b-58cc-4372-a567-0e02b2c3d479", "site_id": "111e8400-
e29b-41d4-a716-446655440111", "description": "Oil spill observed near pump P-302. Secondary
containment holding, but primary seal is leaking.", "risk_score": 4, "local_timestamp":
"2026-10-07T14:32:00Z" } ] }
```

Django Conflict Resolution Logic

When the Django API processes the list, it should utilize an `update_or_create` logic based on the `report_id`. If a network drops during the sync response, the mobile app might retry sending a payload that was already successfully saved in PostgreSQL. Idempotency is critical to avoid duplicate incident reports skewing HSE data metrics.

4. .NET MAUI Client Setup

AI INSTRUCTION: Generate the frontend code using C# and XAML in .NET MAUI. The application requires three core services via Dependency Injection: a LocalDatabaseService (managing SQLite), a SyncService, and a NetworkConnectivityService (using `Microsoft.Maui.Networking`).



Local SQLite Implementation

Use `sqlite-net-pcl` for local data modeling. When mapping the model in C#, ensure the `[PrimaryKey]` attribute is applied to a string or Guid field. The save method must explicitly set `IsSynced = false` by default.

Network Listener Logic

Implement an event listener hooked to `Connectivity.ConnectivityChanged`. When the state changes to `NetworkAccess.Internet`, the app should fire a background Task that calls `SyncService.SyncPendingReportsAsync()`. Ensure UI thread is not blocked during this background operation. Use `ActivityIndicator` visually if the user is on the history tab during a sync event.

> **Authentication handling:** Do not hardcode credentials. For the sake of the portfolio project, mock an initial login screen that retrieves a JWT token and stores it using .NET MAUI's `SecureStorage.SetAsync("auth_token", token)`. Read this token to attach to the header of the sync POST request.

5. AI Execution Checklist

AI INSTRUCTION: Execute the code generation sequentially. Do not move to the next phase until the previous is successfully generated and verified.

Phase	Component	Success Criteria
Phase 1	Django Models & Serializers	UUID fields present; JSON schema explicitly matches requirements.
Phase 1b	Django Sync Endpoint View	Accepts list, loops objects, utilizes `update_or_create`, handles Exceptions gracefully.
Phase 2	MAUI SQLite Models	`[PrimaryKey]` on ReportID string; `IsSynced` boolean mapped properly.
Phase 3	MAUI UI Layouts (XAML)	Clean layout referencing Shell's operational hazard forms. Picker for severity.
Phase 4	Sync Engine Logic (C#)	Fires automatically on Network connection. Flags SQLite records as `IsSynced = true` on 200 OK.

Shell Interview Presentation Strategy

When presenting the output of this project in Port Harcourt, you must bridge the technical code to Shell's business outcomes. The key phrase to utilize is **"Data-Driven Goal Zero."**

Explain that offshore connectivity on platforms (like the Bonga FPSO) can drop due to weather or technical faults. By engineering an "Offline-First" application, you are guaranteeing that field workers never have an excuse to avoid logging a hazard, directly supporting Shell's safety mandate. The technology (UUIDs, background syncs, REST APIs) is purely a vehicle to ensure corporate safety compliance.

> **Final Agent Directive:** Generate the complete `models.py`, `views.py`, and `serializers.py` for Django first. Then generate the C# SQLite Service class. Output them as clean, well-commented code blocks.